



Introduction to Programming Language: Python

Begin your journey into the world of programming by learning one of the most popular languages: Python. From syntax to data types, Python is easy to learn and rewarding to master.

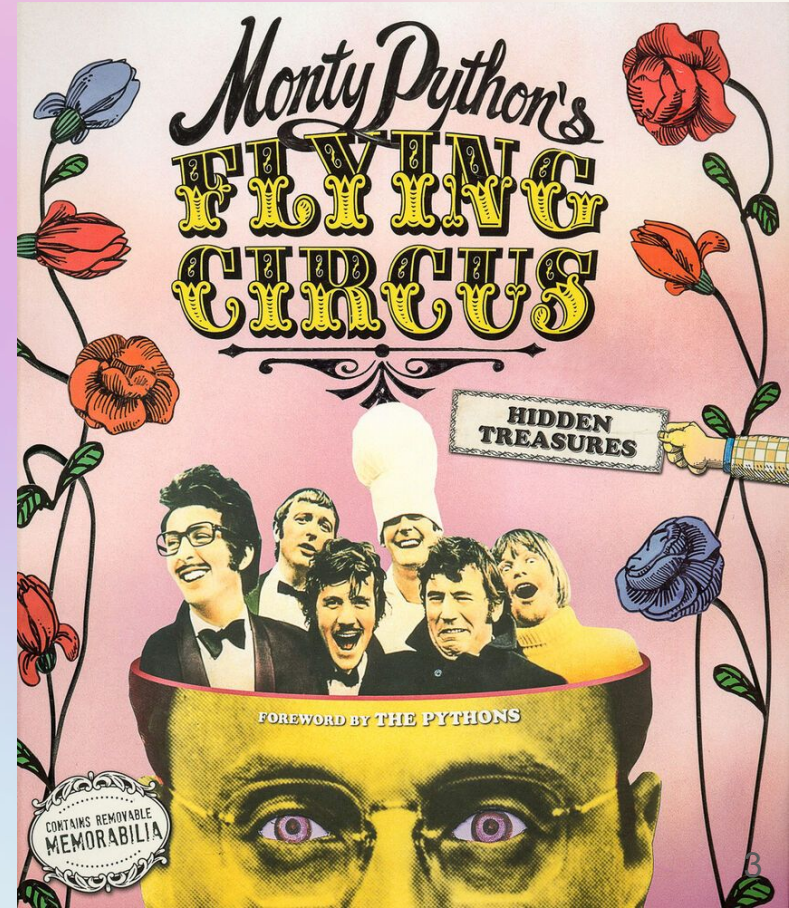
Table of Contents

- 1. Introduction to Python**
- 2. Virtual Environment and Package Management**
- 3. Introduction to Numpy and Pandas**
- 4. Data visualization with Matplotlib and Seaborn**
- 5. Practical Exercises**

1. Introduction to Python

1.1. What is Python?

- Python is a **high-level, interpreted programming language**.
- Created in the late 1980s by **Guido van Rossum** and first released in **1991**.
- The name "Python" was inspired by the British comedy series "**Monty Python's Flying Circus**" of which van Rossum was a fan.
- Major revisions: **Python 1.0** (1994), **Python 2.0** (2000), **Python 3.0** (2008)
- Python 2.x and 3.x coexisted for a significant period due to the changes in syntax and functionality between the versions.
- **Python 2 reached its end-of-life on January 1, 2020**, and since then, Python 3 has been the recommended and actively developed version.



1. Introduction to Python

1.2. Features and Benefits

1

Readable and Simple Syntax

Python has indentation-based structure, clean syntax makes it easy to read and write code, even for beginners.

2

Large Standard Library

A vast collection of libraries and modules, offering ready-to-use functions without requiring additional installations.

3

Platform Independence

Highly portable, running on various platforms such as Windows, macOS, Linux, and more without modification.

4

Community Support

Python has a large and active community of developers who contribute to its development and provide resources.

1. Introduction to Python

1.3. Data Types

Numbers

Supports integers, floating-point numbers and complex numbers.

List/Tuple/Set

Used to store multiple data at once.

Strings

Ordered sequences of characters.

Dictionary

A collection that allows us to store data in key-value pairs.

1. Introduction to Python

1.3. Data Types

1.3.1. Number

- The number data types are used to store the numeric values.
- Python supports **integers**, **floating-point numbers** and **complex numbers**:
 - int** - holds signed integers of non-limited length.
 - float** - holds floating decimal points, up to 15 decimal places.
 - complex** - holds complex numbers.
- We can also use built-in functions like `int()`, `float()` and `complex()` to convert between types explicitly. These functions can even convert from strings.

1. Introduction to Python

1.3. Data Types

1.3.2. Strings

- We use **single quotes** ('...') or **double quotes** ("...") to represent a string in Python.
- Python Strings are **immutable**.
- Python provides a rich set of operations and methods to manipulate and work with strings.

1. Introduction to Python

1.3. Data Types

1.3.3. List/Tuple/Set

- **Lists** are used to store multiple data at once.
- **Tuple** is similar to a list but **cannot change the elements once it is assigned**.
- **Set** is a collection of unique data **which its elements cannot be duplicate**.

1. Introduction to Python

1.3. Data Types

1.3.4. Dictionary

- In Python, a dictionary is a collection that allows us to store data in **key-value pairs**.
- We create dictionaries by placing key:value pairs inside curly brackets, separated by commas.
- Python dictionaries are **mutable** (changeable). We can change the value of a dictionary element by referring to its key.

1. Introduction to Python

1.4. Flow Control

Conditional Statements

Execute code based on certain conditions, using if, elif, and else statements.

Loops

Execute a block of code repeatedly, either a set number of times (for loop) or until a certain condition is met (while loop).

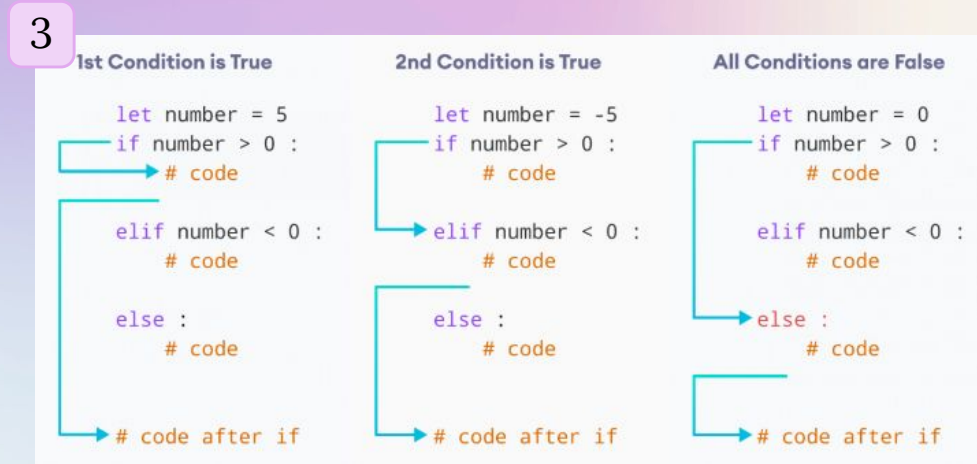
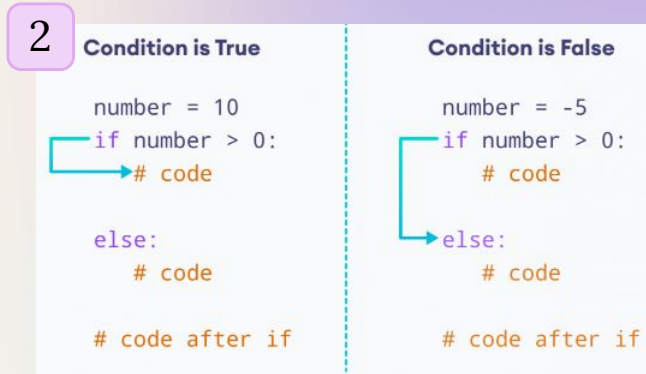
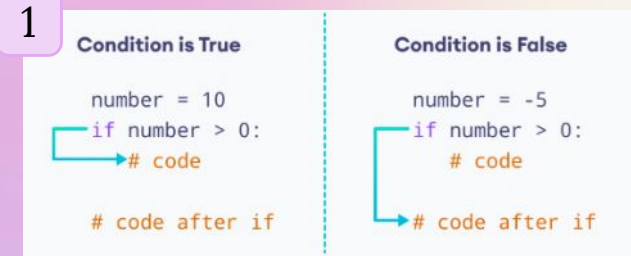
1. Introduction to Python

1.4. Flow Control

1.4.1. Conditional Statements

In Python, there are three forms of the if...else statement.

- (1) if statement
- (2) if...else statement
- (3) if...elif...else statement

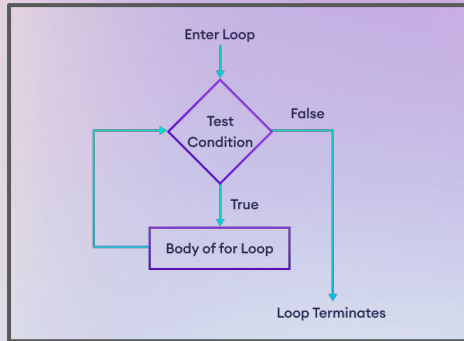


1. Introduction to Python

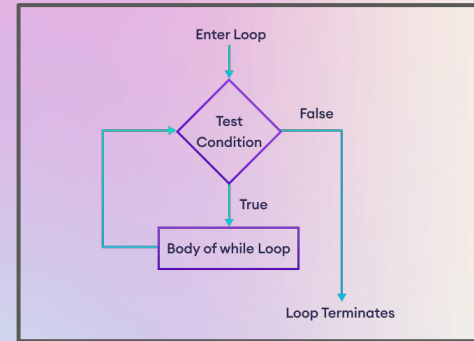
1.4. Flow Control

1.4.2. Loops

A **for** loop is used to iterate over sequences such as lists, tuples, string, etc.



A **while** loop is used to run a block code until a certain condition is met.

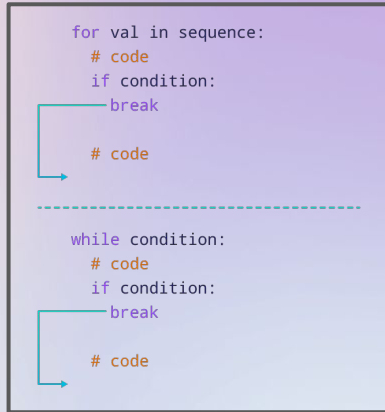


1. Introduction to Python

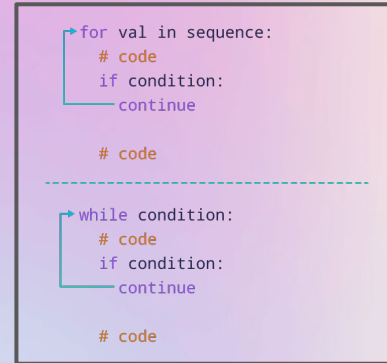
1.4. Flow Control

1.4.2. Loops

The break statement is used to terminate the loop immediately when it is encountered.



The continue statement is used to skip the current iteration of the loop and go to the next iteration.



1. Introduction to Python

1.5. Function

- A **function** is a block of code that performs a specific task.
- There are two types of function in Python programming:
 - **Standard library functions** - These are built-in functions in Python that are available to use.
Example: `print()`, `type()`, `dir()`,...
 - **User-defined functions** - We can create our own functions based on our requirements.

1. Introduction to Python

1.6. Classes

- Python is a versatile programming language that supports various programming styles, including object-oriented programming (OOP) through the use of objects and classes.
- Some key features:
 - **Inheritance:** a way of creating a new class for using details of an existing class without modifying it.
 - **Encapsulation:** refers to the bundling of attributes and methods inside a single class. It prevents outer classes from accessing and changing attributes and methods of a class. In Python, we denote private attributes using underscore as the prefix i.e single `_` or double `__`.
 - **Polymorphism:** the same entity (method or operator or object) can perform different operations in different scenarios.

1. Introduction to Python

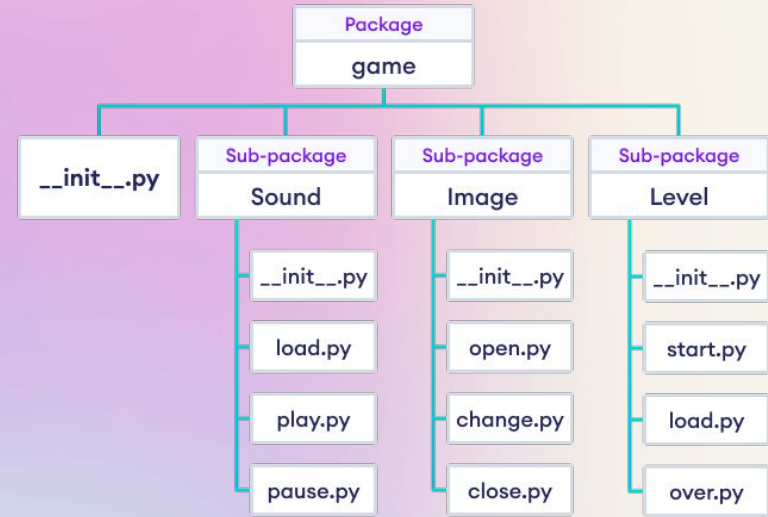
1.7. Modules

- Module is a file that contains code to perform a specific task. A module may contain variables, functions, classes etc.
- The Python standard library contains well over **200 modules**. We can import a module according to our needs.
- Using the module name we can access the function using the dot (.) operator.

1. Introduction to Python

1.8. Packages

- A package is a container that contains various functions to perform specific tasks.
- In Python, we can import modules from packages using the dot (.) operator.
- The `__init__.py` used to define package-level attributes and operations, executed when the package or module is imported.
- In Python 3.3 and later versions, the `__init__.py` file is not required to define a package. If a directory contains modules but lacks an `__init__.py` file, it's still considered a package.



2. Virtual Environment & Package Management

2.1. Virtual Environment

- A virtual environment is a self-contained directory that houses a specific Python interpreter version and its libraries. It allows developers to work on multiple projects with different dependencies without conflicts.
- Why use virtual environments?
 - o **Dependency isolation:** Ensures that each project can have its own set of dependencies.
 - o **Reproducibility:** Makes it easier to replicate the development environment on different machines.
 - o **Dependency version control:** Helps manage and track specific versions of libraries required for a project.
- Popular tools for creating virtual environments: **virtualenv**, **conda**, **venv**,...

2. Virtual Environment & Package Management

2.2. Package Management

- Package management involves installing, upgrading, configuring, and removing software packages and their dependencies.
- Why is package management important?
 - **Dependency resolution:** Ensures that all necessary components for a software package are installed correctly.
 - **Version control:** Helps manage different versions of packages used in a project.
 - **Ease of distribution:** Simplifies sharing software across different environments.
- Popular Python package managers:
 - **pip:** The default package manager for Python. It installs Python packages from the Python Package Index (PyPI).
 - **conda:** It can manage both Python and non-Python packages.

3. Introduction to Numpy and Pandas

3.1. Numpy

- NumPy is a fundamental Python library for numerical computing that provides support for arrays, matrices, and a wide array of mathematical functions to operate on these data structures.
- Key features of NumPy:
 - **N-dimensional Array Object:** NumPy is a **multi-dimensional array** that can store elements of the same data type. These arrays are allowed for vectorized operations.
 - **Array Operations:** NumPy offers a vast array of mathematical operations that can be performed directly on arrays **without the need for explicit looping**, known as **vectorization**.
 - **Broadcasting:** NumPy 's broadcasting capability allows arrays with different shapes to be combined in arithmetic operations.
 - **Integration with other Libraries:** NumPy is often used in conjunction with other libraries like SciPy (scientific computing), Matplotlib (plotting), Pandas (data manipulation), and others, forming the basis of the scientific Python ecosystem.

3. Introduction to Numpy and Pandas

3.2. Pandas

- Pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language.
- Key Features of Pandas:
 - **DataFrame Object:** The core of Pandas is the DataFrame, a two-dimensional labeled data structure similar to a table or spreadsheet. It consists of rows and columns, allowing easy manipulation and analysis of data.
 - **Data Exploration and Analysis:** It offers powerful methods for grouping, aggregating, sorting, filtering, and analyzing data.
 - **Input/Output Operations:** Pandas supports reading and writing data from various file formats such as CSV, Excel, SQL databases, JSON, and more.
 - **Integration with Other Libraries:** Pandas integrates well with other Python libraries like NumPy, Matplotlib, and Scikit-learn, forming a robust ecosystem for data analysis and machine learning.

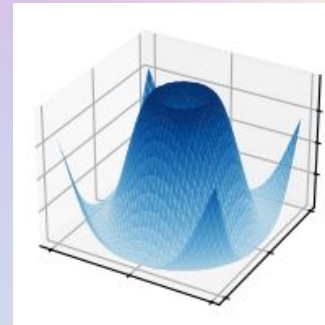
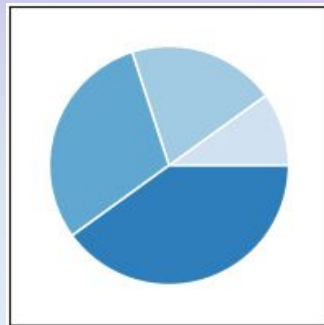
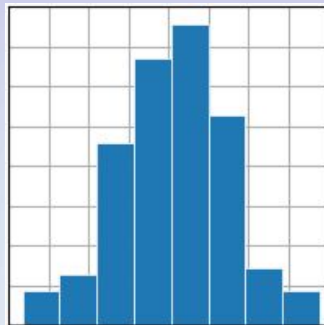
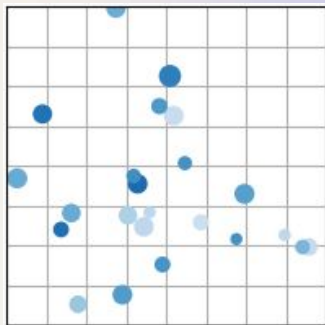
4. Data visualization with Matplotlib & Seaborn

4.1. Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.

Matplotlib makes easy things easy and hard things possible.

- Create publication quality plots.
- Make interactive figures that can zoom, pan, update.
- Customize visual style and layout.
- Export to many file formats.
- Embed in JupyterLab and Graphical User Interfaces.
- Use a rich array of third-party packages built on Matplotlib.



4. Data visualization with Matplotlib & Seaborn

4.1. Seaborn

Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.

